



International University of Technology
Twintech

Visual Programming

C#

By. Mohammed Talat

By.
Mr. Mohammed Talat

LAB EVALUATION

Student Name:-----

Department:----- Group Number:-----

ID	Lab Topics	Date of Conducted	Programs / Activity	Assignments	Signature
1		/ /			
2		/ /			
3		/ /			
4		/ /			
5		/ /			
6	Rest of Infrastructure layer and introduction to application layer	/ /			
7		/ /			
8		/ /			
9		/ /			
10		/ /			

Student Responsibilities:

- 1) **Lab Sheet Sign-off:** Students are required to obtain a signature on all lab sheets before leaving the lab. Failure to do so will result in a **zero** grade for the respective lab.
- 2) **Independent Work:** All program outputs and assignments must be completed independently by each student.
- 3) Students are responsible for maintaining the lab manual in a clean, safe, and organized condition.
- 4) **Lab Manual Submission:** The entire lab manual must be submitted in person during Week 11.- *any student failed to do so will get ZERO in the final mark.*



International University of Technology
Twintech

Rest of Infrastructure layer and introduction to application layer

" Bugs are just features waiting to be fixed. Keep going!"

1.0 Objectives

After completing this lab, students will be able to:

- **Implement** repository interfaces within the Infrastructure layer.
- **Use** Entity Framework Core (AppDbContext) to perform CRUD operations.
- **Understand** the role of the Application Layer in Clean Architecture.
- **Create** the Application Class Library project.
- **Design** Data Transfer Objects (DTOs) for safe data transfer.
- **Define** Service Interfaces to outline application use cases.

1.1 Introduction

Briefly explain:

- In the previous labs, we defined our business entities and repository interfaces in the Domain layer.
- In this lab, we will first complete the Infrastructure layer by creating the **Repository Implementation** classes that talk directly to the database via EF Core.
- Then, we will transition to the **Application Layer**, where we will define how data enters/leaves the application using **DTOs**, and declare our business features using **Service Interfaces**.

2.0 Repository Implementation

The Repository Implementation contains the actual database query logic. It implements the interfaces from the Domain layer and uses ApplicationDbContext to interact with the database tables.

```
public class UserRepository : IUserRepository
{
    private readonly ApplicationDbContext _context;

    public UserRepository(ApplicationDbContext context)
    {
        _context = context;
    }

    public void Add(User user)
    {
        _context.Users.Add(user);
        _context.SaveChanges();
    }

    public User GetById(int id)
    {
        return _context.Users.Find(id);
    }

    public IEnumerable<User> GetAll()
    {
        return _context.Users.ToList();
    }
}
```

3.0 Introduction to the Application Layer

The **Application Layer** sits right above the Domain layer and orchestrates the system's business logic. It contains specific use cases (e.g., creating an account, processing an order).

In this lab, we focus on its two core introductory components:

1. **DTOs (Data Transfer Objects):** Simple objects holding data, preventing exposure of domain entities.
2. **Service Interfaces:** Contracts defining what operations the application layer can perform.

4.0 Data Transfer Objects (DTOs)

A **DTO** is a pure data contract with no business logic. We use it to receive requests from the user or send responses back.

Why use DTOs?

- **Security:** Prevents clients from seeing or modifying sensitive entity properties (like passwords or hidden fields).
- **Decoupling:** Decouples the presentation layer from the database schema.

Example: UserDTO

```
// Application Layer
public class UserDto
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
}
```

5.0 Service Interfaces

A **Service Interface** defines the available operations (use cases) for a specific feature. It uses the DTOs we created to accept input and return output. It only specifies *what* the application can do.

Example: IUserService

```
public interface IUserService
{
    void CreateUser(UserDto userDto);
    UserDto GetUserById(int id);
    IEnumerable<UserDto> GetAllUsers();
}
```

Assignment

1. Complete all **Repository Implementations** for your Final Project using EF Core.
2. Create the **Application Layer** Class Library project.
3. Design and implement **DTO classes** that exactly mirror your Domain Entities.
4. Define the **Service Interfaces** (I...Service) outlining the primary operations (Create, Get, etc.) using your DTOs.